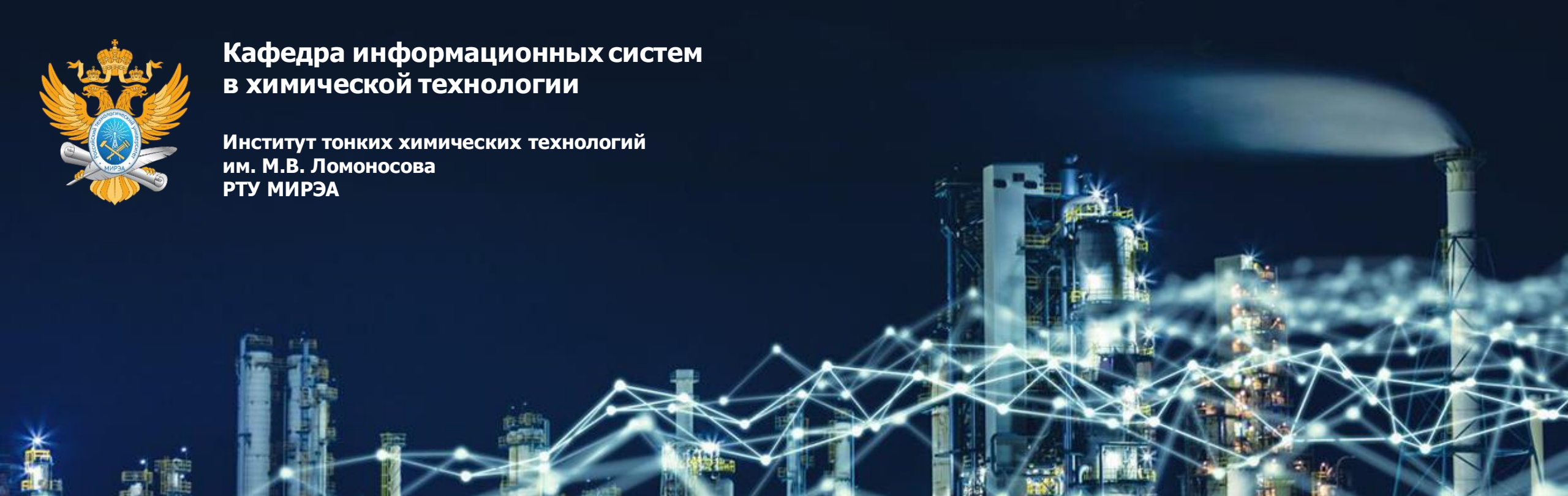




**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХИМИЧЕСКОЙ ТЕХНОЛОГИИ



Лекция 8. Пользователи и права доступа

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- Управление локальными учетными записями пользователей
- Управление локальными группами
- Управление паролями пользователей
- Модель контроля доступа к файлам в Linux: типы разрешений
- Управление разрешениями на файлы
- Управление разрешениями по умолчанию и доступом к файлам

Концепция пользователей и групп в Linux

-  Получение привилегий суперпользователя
-  Управление локальными учетными записями пользователей
-  Управление локальными группами
-  Управление паролями пользователей
-  Модель контроля доступа к файлам в Linux: типы разрешений
-  Управление разрешениями на файлы
-  Управление разрешениями по умолчанию и доступом к файлам

Что такое пользователь?

Учётные записи пользователей используются для выполнения команд, программ, совместного доступа к ресурсам и т.д.

Типы учётных записей

- Суперпользователь (superuser)
- Системные учётные записи
- Обычные пользователи

id – команда для просмотра информации о пользователе

- информация о текущем пользователе (команда **id** без аргументов)

```
[user01@host ~]$ id
uid=1000(user01) gid=1000(user01) groups=1000(user01)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

- информация о другом пользователе (имя пользователя в качестве аргумента для команды **id**)

```
[user01@host]$ id user02
uid=1002(user02) gid=1001(user02) groups=1001(user02)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Команды управления пользователями и процессами

 **ls -l** – команда для просмотра владельца файла

```
[user01@host ~]$ ls -l file1
-rw-rw-r--. 1 user01 user01 0 Feb 5 11:10 file1
[user01@host ~]$ ls -ld dir1
drwxrwxr-x. 2 user01 user01 6 Feb 5 11:10 dir1
```

 **ps** – команда для просмотра процессов и их владельцев

```
[user01@host ~]$ ps -au
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
Root	777	0.0	0.0	225752	1496	tty1	Ss+	11:03	0:00	/sbin/agetty -o
-p -- \u --noclear tty1 linux										
Root	780	0.0	0.1	225392	2064	ttyS0	Ss+	11:03	0:00	/sbin/agetty -o
-p -- \u --keep-baud 115200,38400,9600										
User01	1207	0.0	0.2	234044	5104	pts/0	Ss	11:09	0:00	-bash
User01	1319	0.0	0.2	266904	3876	pts/0	R+	11:33	0:00	ps au

Файл /etc/passwd

- Выходные команды `ps` отображают пользователей по именам, но внутри операционная система использует `UID` для отслеживания пользователей. Сопоставление имен пользователей с `UID` определяется в базах данных информации об учётных записях пользователей.
- По умолчанию системы используют файл `/etc/passwd` для хранения информации о локальных пользователях.
 - Каждая строка в файле `/etc/passwd` содержит информацию об одном пользователе. Он разделен на семь полей, разделенных двоеточием.
 - ①user01: ②x: ③1000: ④1000: ⑤User One: ⑥/home/user01: ⑦/bin/bash

Что такое группа?

- **Группа** – это совокупность пользователей, которым необходим общий доступ к файлам и другим системным ресурсам.
 - Группы можно использовать для предоставления доступа к файлам группе пользователей, а не только одному пользователю.
- Как и у пользователей, у групп есть имена, чтобы с ними было легче работать.
 - Внутри система различает группы по присвоенному им уникальному идентификационному номеру, идентификатору группы или GID.
 - Сопоставление имен групп с GID определяется в базах данных информации об учетных записях групп.
- По умолчанию системы используют файл `/etc/group` для хранения информации о локальных группах.
 - Каждая строка в файле `/etc/group` содержит информацию об одной группе. Каждая запись группы разделена на четыре поля, разделенные двоеточием.
 - ①group01: ②x: ③10000: ④user01,user02,user03
- **Основные и дополнительные группы**
 - У каждого пользователя есть ровно одна основная группа.
 - Пользователи также могут иметь дополнительные группы.

Основные группы и дополнительные группы

- У каждого пользователя есть ровно одна основная группа. Для локальных пользователей это группа, указанная по номеру GID в файле `/etc/passwd`. По умолчанию это группа, которая будет владеть новыми файлами, созданными пользователем.
 - Обычно, когда вы создаете нового обычного пользователя, создается новая группа с тем же именем, что и у этого пользователя. Эта группа используется в качестве основной группы для нового пользователя, и этот пользователь является единственным членом этой частной группы пользователей. Это помогает сделать управление разрешениями на файлы более простым.
- Пользователи также могут иметь дополнительные группы. Членство в дополнительных группах определяется файлом `/etc/group`. Пользователям предоставляется доступ к файлам в зависимости от того, имеет ли доступ какая-либо из их групп. Неважно, являются ли группа или группы, имеющие доступ, основными или дополнительными для пользователя.
 - Например, если у пользователя `user01` есть основная группа `user01` и дополнительные группы `wheel` и `webadmin`, то этот пользователь может читать файлы, доступные для чтения любой из этих трёх групп.

- Концепция пользователей и групп в Linux
- **Получение привилегий суперпользователя**
- Управление локальными учетными записями пользователей
- Управление локальными группами
- Управление паролями пользователей
- Модель контроля доступа к файлам в Linux: типы разрешений
- Управление разрешениями на файлы
- Управление разрешениями по умолчанию и доступом к файлам

Получение привилегий суперпользователя

Что такое суперпользователь (root)?

- В большинстве операционных систем есть своего рода суперпользователь – пользователь, обладающий всей властью над системой.
 - В Astra Linux это пользователь `root`. Этот пользователь имеет право переопределять обычные привилегии в файловой системе и используется для управления и администрирования системы. Для выполнения таких задач, как установка или удаление программного обеспечения, а также для управления системными файлами и каталогами, пользователи должны повысить свои привилегии до пользователя `root`.

Переключение между учётными записями, команда **su**

- Команда `su` позволяет пользователям переключаться на другую учётную запись пользователя. Если вы запускаете `su` из учётной записи обычного пользователя, вам будет предложено ввести пароль учётной записи, на которую вы хотите переключиться. Когда `root` запускает `su`, вам не нужно вводить пароль пользователя.

```
[user01@host ~]$ su - user02
```

```
Password:
```

```
[user02@host ~]$
```

- Если вы не введёте имя пользователя, команда `su` или `SU` - попытается по умолчанию переключиться на `root`.

```
[user01@host ~]$ su -
```

```
Password:
```

```
[root@host ~]#
```

Получение привилегий суперпользователя

su -c – аналог Run As Administrator (windows)

- Учетная запись `root` в Linux примерно эквивалентна учетной записи локального администратора в Microsoft Windows. В Linux большинство системных администраторов входят в систему как непривилегированный пользователь и используют различные инструменты для временного получения `root`-прав.
- Команда `su` чаще всего используется для получения интерфейса командной строки (приглашения оболочки), который работает от имени другого пользователя, обычно `root`. Однако с опцией `-c` её можно использовать как утилиту Windows `runas` для запуска произвольной программы от имени другого пользователя.

Выполнение команд с помощью `sudo`

- В некоторых случаях учётная запись пользователя `root` может вообще не иметь действительного пароля по соображениям безопасности.
- В этом случае пользователи не могут войти в систему как `root` напрямую с паролем, а `su` нельзя использовать для получения интерактивной оболочки. Одним из инструментов, который можно использовать для получения `root`-доступа в этом случае, является `sudo`

Получение привилегий суперпользователя

Использование sudo вместо su

- В отличие от su, sudo обычно требует от пользователей ввода собственного пароля для аутентификации, а не пароля учетной записи пользователя, к которой они пытаются получить доступ. То есть пользователям, использующим sudo для запуска команд от имени root, не нужно знать пароль root. Вместо этого они используют свои собственные пароли для аутентификации доступа.
- Кроме того, sudo можно настроить так, чтобы определенные пользователи могли выполнять любую команду от имени другого пользователя или только некоторые команды от имени этого пользователя.
- Например, если sudo настроен так, чтобы разрешить пользователю user01 запускать команду usermod от имени root, user01 может выполнить следующую команду, чтобы заблокировать или разблокировать учётную запись пользователя:

```
[user01@host ~]$ sudo usermod -L user02
[sudo] password for user01:
[user01@host ~]$ su - user02
Password:
su: Authentication failure
[user01@host ~]$
```

Получение привилегий суперпользователя

Настройка sudo

- Основной файл конфигурации для sudo – `/etc/sudoers`. Во избежание проблем, если несколько администраторов попытаются отредактировать его одновременно, его следует редактировать только с помощью специальной команды `visudo`. Например, следующая строка из файла `/etc/sudoers` разрешает доступ sudo для членов группы `wheel`.

```
%wheel ALL=(ALL) ALL
```

В этой строке `%wheel` – это пользователь или группа, к которым применяется правило. `%` указывает, что это группа. `ALL=(ALL)` указывает, что на любом хосте, на котором может быть этот файл, группа может запускать любую команду. Последнее `ALL` указывает, что группа может запускать эти команды от имени любого пользователя в системе.

- По умолчанию `/etc/sudoers` также включает содержимое любых файлов в каталоге `/etc/sudoers.d` как часть файла конфигурации. Это позволяет администратору добавить доступ к sudo для пользователя, просто поместив соответствующий файл в этот каталог.
- Использование дополнительных файлов в каталоге `/etc/sudoers.d` удобно и просто. Вы можете включить или отключить доступ к sudo, просто скопировав файл в каталог или удалив его из каталога.

Получение привилегий суперпользователя

🔄 `sudo su` - и `sudo -i`.

- Обе команды работают, но между ними есть некоторые тонкие различия.
 - Команда `sudo su` - настраивает корневую среду точно так же, как при обычном входе в систему, поскольку команда `su` - игнорирует настройки, сделанные `sudo`, и настраивает среду с нуля.
 - Конфигурация по умолчанию команды `sudo -i` на самом деле настраивает некоторые детали среды пользователя `root` иначе, чем при обычном входе в систему. Например, он устанавливает переменную среды `PATH` немного по-другому. Это влияет на то, где оболочка будет искать команды.
- Вы можете сделать так, чтобы `sudo -i` вел себя как `su`, отредактировав `/etc/sudoers` с помощью `visudo`.
 - Найдите строку
`Defaults secure_path = /sbin:/bin:/usr/sbin:/usr/bin`
 - И замените следующими записями:
`Defaults secure_path = /usr/local/bin:/usr/bin`
`Defaults>root secure_path = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin`

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- **Управление локальными учётными записями пользователей**
- Управление локальными группами
- Управление паролями пользователей
- Модель контроля доступа к файлам в Linux: типы разрешений
- Управление разрешениями на файлы
- Управление разрешениями по умолчанию и доступом к файлам

Создание пользователей из командной строки

- Команда `useradd username` создает нового пользователя. Она создаёт домашний каталог пользователя и добавляет информацию об учётной записи, а также создаёт группу для пользователя с именем пользователя. По умолчанию не установлен действительный пароль, и пользователь не может войти в систему, пока не будет установлен пароль.
- Команда `useradd --help` отображает основные параметры, которые можно использовать для переопределения значений по умолчанию. В большинстве случаев те же параметры можно использовать с командой `usermod` для изменения существующего пользователя.
- Некоторые значения по умолчанию, такие как диапазон UID и правила устаревания пароля по умолчанию, считываются из файла `/etc/login.defs`. Значения в этом файле используются только при создании новых пользователей. Изменение этого файла не влияет на существующих пользователей.

Изменение существующих пользователей из командной строки

- Команда `usermod --help` отображает основные параметры, которые можно использовать для изменения учётной записи.

Управление локальными пользователями

Наиболее популярные параметры usermod:

ОПЦИИ usermod	ПРИМЕНЕНИЕ
-c, --comment COMMENT	Добавить полное имя пользователя в поле комментария
-g, --gid GROUP	Определить основную группу для учётной записи пользователя
-G, --groups GROUPS	Определить список дополнительных групп для учётной записи пользователя, разделённых запятыми
-a, --append	Используется с параметром -G для добавления дополнительных групп к текущему набору членства пользователя в группах вместо замены набора дополнительных групп новым набором
-d, --home HOME_DIR	Определить конкретный домашний каталог для учётной записи пользователя
-m, --move-home	Переместить домашний каталог пользователя в новое место. Должен использоваться с опцией -d
-s, --shell SHELL	Определить конкретную оболочку входа для учётной записи пользователя
-L, --lock	Заблокировать учётную запись пользователя
-U, --unlock	Разблокировать учётную запись пользователя

🔄 Удаление пользователей из командной строки

- Команда `userdel username` удаляет сведения об имени пользователя из `/etc/passwd`, но оставляет домашний каталог пользователя.
- Команда `userdel -r username` удаляет сведения об имени пользователя из `/etc/passwd`, а также удаляет домашний каталог пользователя.
 - Когда пользователь удаляется с помощью `userdel` без указания параметра `-r`, в системе останутся файлы, принадлежавшие его UID. Это также может произойти, когда файл, владельцем которого является удалённый пользователь, существует за пределами домашнего каталога этого пользователя. Это может привести к утечке информации и другим проблемам с безопасностью.
 - В Astra Linux команда `useradd` присваивает новым пользователям первый свободный UID, больший или равный 1000, если вы явно не укажете его с помощью параметра `-u`. Вот как может происходить утечка информации. Если первый свободный UID ранее был назначен учётной записи пользователя, которая с тех пор была удалена из системы, UID старого пользователя будет переназначен новому пользователю, давая новому пользователю право владения оставшимися файлами старого пользователя.

🔄 Установка паролей из командной строки

- Команда `passwd username` устанавливает начальный пароль или изменяет существующий пароль для пользователя.
- Пользователь `root` может установить любой пароль для любого пользователя. Если пароль не соответствует минимальным рекомендуемым критериям, отображается сообщение, но затем следует запрос на повторный ввод нового пароля и все токены успешно обновляются.

```
[root@host ~]# passwd user01
```

```
Changing password for user user01.
```

```
New password: redhat
```

```
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
```

```
Retype new password: redhat
```

```
passwd: all authentication tokens updated successfully.
```

```
[root@host ~]#
```

- Обычный пользователь должен выбрать пароль длиной не менее восьми символов, не основанный на словарном слове, а также не должен повторять имя пользователя или предыдущие пароли.

Диапазоны UID

Определенные номера UID и диапазоны номеров используются Astra Linux для определенных целей:

- UID 0 – всегда назначается учетной записи суперпользователя root.
- UID 1-200 – это диапазон «системных пользователей», статически назначаемых системным процессам Astra Linux.
- UID 201-999 – это диапазон «системных пользователей», используемых системными процессами, не владеющими файлами в файловой системе. Обычно они назначаются динамически из доступного пула, при установке программного обеспечения. Программы запускаются от имени этих «непривилегированных» системных пользователей, чтобы ограничить их доступ только к тем ресурсам, которые им необходимы для работы.
- UID 1000+ – диапазон, доступный для назначения обычным пользователям.

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- Управление локальными учетными записями пользователей
- **Управление локальными группами**
- Управление паролями пользователей
- Модель контроля доступа к файлам в Linux: типы разрешений
- Управление разрешениями на файлы
- Управление разрешениями по умолчанию и доступом к файлам

Создание групп из командной строки

- Команда `groupadd` создает группы. Без опций команда `groupadd` использует следующий доступный GID из диапазона, указанного в файле `/etc/login.defs`, при создании группы.
- Параметр `-g` задает конкретный GID для использования группой.

```
[user01@host ~]$ sudo groupadd -g 10000 group01
```

```
[user01@host ~]$ tail /etc/group
```

```
.....
```

```
group01:x:10000:
```

- Параметр `-r` создает системную группу с использованием GID из диапазона допустимых системных GID, перечисленных в файле `/etc/login.defs`. Элементы конфигурации `SYS_GID_MIN` и `SYS_GID_MAX` в `/etc/login.defs` определяют диапазон системных GID.

```
[user01@host ~]$ sudo groupadd -r group02
```

```
[user01@host ~]$ tail /etc/group
```

```
.....
```

```
group01:x:10000:
```

```
group02:x:988:
```


Изменение существующих групп из командной строки

- Команда `groupmod` изменяет свойства существующей группы. Параметр `-n` указывает новое имя для группы.

```
[user01@host ~]$ sudo groupmod -n group0022 group02
```

```
[user01@host ~]$ tail /etc/group
```

```
...output omitted...
```

```
group0022:x:988:
```

- Обратите внимание, что имя группы обновлено: `group0022` вместо `group02`.

- Параметр `-g` указывает новый GID.

```
[user01@host ~]$ sudo groupmod -g 20000 group0022
```

```
[user01@host ~]$ tail /etc/group
```

```
...output omitted...
```

```
group0022:x:20000:
```

- Обратите внимание, что GID обновлён: `20000` вместо `988`.

Управление локальными группами

Удаление групп из командной строки

- Команда `groupdel` удаляет группы.

```
[user01@host ~]$ sudo groupdel group0022
```

Изменение членства в группе из командной строки

Членство в группе контролируется с помощью управления пользователями.

- Используйте команду `usermod -g`, чтобы изменить основную группу пользователя.

```
[user01@host ~]$ id user02
```

```
uid=1006(user02) gid=1008(user02) groups=1008(user02)
```

```
[user01@host ~]$ sudo usermod -g group01 user02
```

```
[user01@host ~]$ id user02
```

```
uid=1006(user02) gid=10000(group01) groups=10000(group01)
```

- Используйте команду `usermod -aG`, чтобы добавить пользователя в дополнительную группу.

```
[user01@host ~]$ id user03
```

```
uid=1007(user03) gid=1009(user03) groups=1009(user03)
```

```
[user01@host ~]$ sudo usermod -aG group01 user03
```

```
[user01@host ~]$ id user03
```

```
uid=1007(user03) gid=1009(user03) groups=1009(user03),10000(group01)
```

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- Управление локальными учетными записями пользователей
- Управление локальными группами
- **Управление паролями пользователей**
- Модель контроля доступа к файлам в Linux: типы разрешений
- Управление разрешениями на файлы
- Управление разрешениями по умолчанию и доступом к файлам

Управление паролями пользователей

- В своё время зашифрованные пароли хранились в доступном и легко читаемом файле `/etc/passwd`. Считалось, что это достаточно безопасно, пока атаки на зашифрованные пароли по словарю не стали обычным явлением. В этот момент зашифрованные пароли были перенесены в отдельный файл `/etc/shadow`, доступный для чтения только пользователю `root`. Этот новый файл также позволил реализовать функции устаревания пароля и истечения срока его действия.

- Как и в файле `/etc/passwd`, у каждого пользователя есть строка в файле `/etc/shadow`.

Пример строки из `/etc/shadow` с девятью полями, разделенными двоеточием:

①user03:②\$6\$CSsX...*output omitted*...:③17933:④0:⑤99999:⑥7:⑦2:⑧18113:⑨(*)

Управление паролями пользователей

Формат зашифрованного пароля

- Поле зашифрованного пароля хранит три части информации: используемый алгоритм хеширования, случайный ключ и зашифрованный хэш. Каждая часть информации разделяется знаком \$.
- \$①6\$②CSsXcYG1L/4ZfHr/\$③2W6evvJahUfzfHpc9X.45Jc6H30E...*output omitted*...(*)

Основная причина объединения ключа с паролем – защита от атак с использованием предварительно вычисленных списков хэшей паролей. Добавление ключей изменяет полученные хэши, делая предварительно созданный список бесполезным. Если злоумышленнику удастся получить копию файла /etc/shadow, использующего ключи, ему все равно потребуется выполнить подбор пароля, что потребует больше времени и усилий.

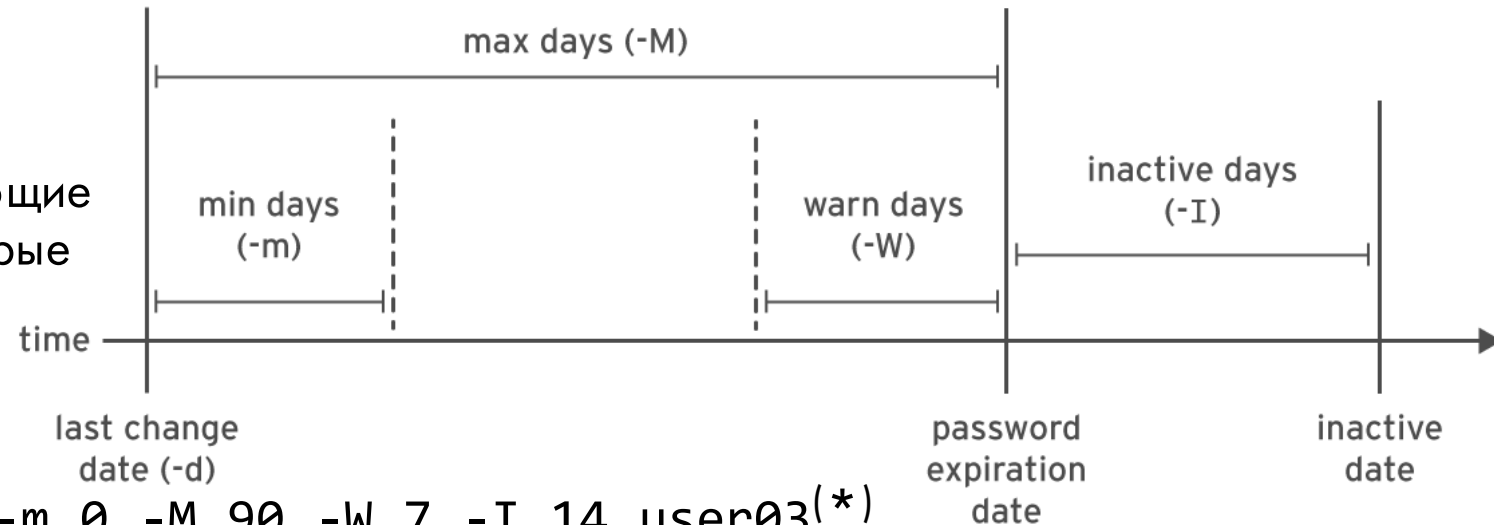
Управление паролями пользователей

Проверка пароля

- Когда пользователь пытается войти в систему, система ищет запись для пользователя в `/etc/shadow`, объединяет ключ пользователя с введённым незашифрованным паролем и шифрует их с использованием указанного алгоритма хеширования. Если результат соответствует зашифрованному хэшу, пользователь ввёл правильный пароль. Если результат не соответствует зашифрованному хэшу, пользователь ввёл неверный пароль и попытка входа не удалась. Этот метод позволяет системе определить, ввёл ли пользователь правильный пароль, не сохраняя этот пароль в форме, пригодной для входа в систему.

Настройка срока действия пароля

- На диаграмме показаны соответствующие параметры устаревания пароля, которые можно настроить с помощью команды `chage` для реализации политики устаревания пароля.



- `[user01@host ~]$ sudo chage -m 0 -M 90 -W 7 -I 14 user03(*)`

Ограничение доступа

- Вы можете использовать команду `chage`, чтобы установить дату истечения срока действия учётной записи. Когда эта дата наступит, пользователь не сможет войти в систему в интерактивном режиме. Команда `usermod` может заблокировать учётную запись с помощью опции `-L`.

```
[user01@host ~]$ sudo usermod -L user03
[user01@host ~]$ su - user03
Password: pass1234
su: Authentication failure
```

- Если пользователь покидает компанию, администратор может заблокировать и установить срок истечения учётной записи с помощью команды `usermod`. Дата должна быть указана в виде количества дней, прошедших с 01.01.1970, или в формате ГГГГ-ММ-ДД.

```
[user01@host ~]$ sudo usermod -L -e 2024-10-05 user03
```

- Команда `usermod` использует параметр `-e` для установки даты истечения срока действия учётной записи для данной учётной записи пользователя. Опция `-L` блокирует пароль пользователя.
- Блокировка учётной записи не позволяет пользователю пройти аутентификацию с помощью пароля в системе. Это рекомендуемый метод предотвращения доступа к учётной записи сотрудника, покинувшего компанию. Если сотрудник вернётся, учётную запись можно будет разблокировать с помощью команды `usermod -U`. Если срок действия учётной записи также истёк, надо изменить дату истечения срока действия.

Оболочка `nologin`

- Оболочка `nologin` действует как заменяющая оболочка для учётных записей пользователей, не предназначенных для интерактивного входа в систему. С точки зрения безопасности разумно отключить учётную запись от входа в систему, когда ей этого не требуется.
- Например, почтовый сервер может потребовать учётную запись для хранения почты и пароль для аутентификации пользователя в почтовом клиенте, используемом для получения почты. Этому пользователю не нужно входить непосредственно в систему. Распространённым решением в этой ситуации является установка оболочки входа пользователя в `/sbin/nologin`. Если пользователь пытается войти в систему напрямую, оболочка `nologin` закроет соединение.

```
[user01@host ~]$ usermod -s /sbin/nologin user03
```

```
[user01@host ~]$ su - user03
```

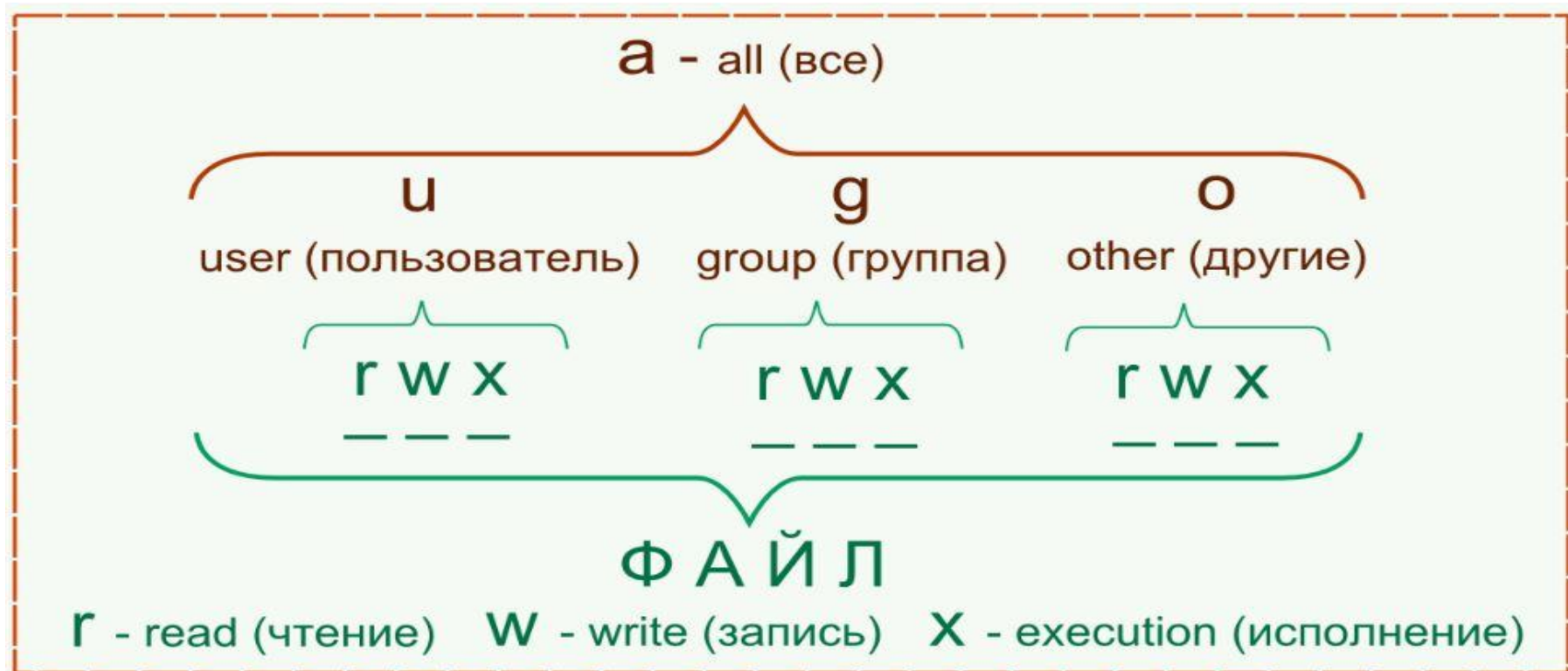
```
Last login: Wed Feb 6 17:03:06 IST 2019 on pts/0
```

```
This account is currently not available.
```

- Оболочка `nologin` предотвращает интерактивное использование системы, но не предотвращает любой доступ. Пользователи могут проходить аутентификацию и загружать или извлекать файлы с помощью таких приложений, как веб-приложения, программы передачи файлов или почтовые программы, если они используют пароль пользователя для аутентификации.

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- Управление локальными учетными записями пользователей
- Управление локальными группами
- Управление паролями пользователей
- **Модель контроля доступа к файлам в Linux: типы разрешений**
- Управление разрешениями на файлы
- Управление разрешениями по умолчанию и доступом к файлам

Модель контроля доступа к файлам в Linux: типы разрешений

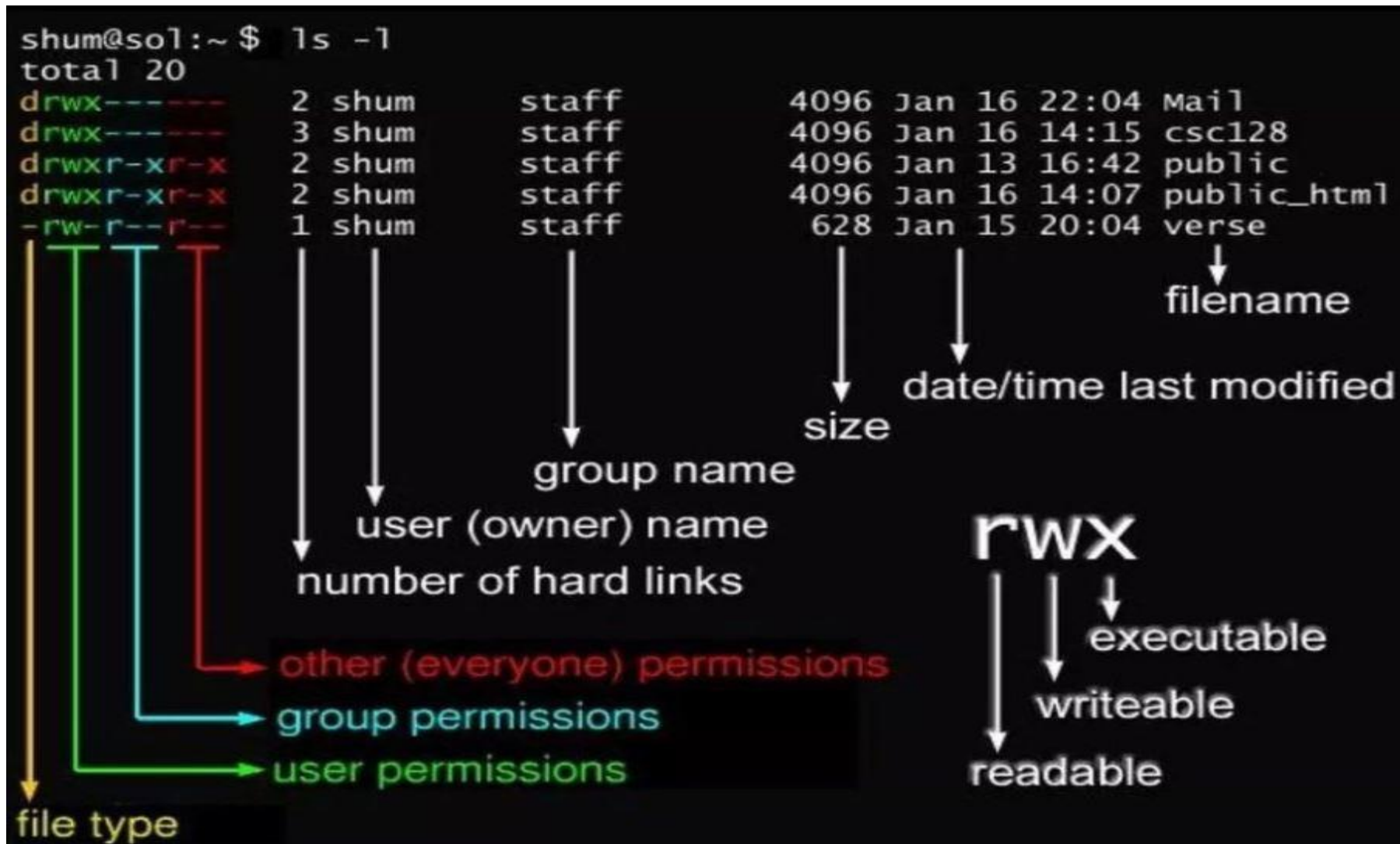


Примеры:

r w - r w - r w -	(все могут читать и изменять)
r w x - - - - -	(полный доступ имеет владелец файла)
r w - r - - r - -	(все могут читать, владелец также изменять)
r w x r - x r - x	(все могут читать и исполнять, владелец также изменять)

Модель контроля доступа к файлам в Linux: типы разрешений

Просмотр разрешения и владения на файл и каталог



Модель контроля доступа к файлам в Linux: типы разрешений

Команды на просмотр разрешения и владения на файл и директорию (каталог)

Параметр `-l` команды `ls` показывает более подробную информацию о разрешениях и владельцах файлов:

```
○ $ ls -l test
```

```
-rw-rw-r--. 1 student student 0 Feb 8 17:36 test
```

Вы можете использовать опцию `-d`, чтобы показать подробную информацию о самом каталоге, а не о его содержимом.

```
○ $ ls -ld /home
```

```
drwxr-xr-x. 5 root root 4096 Jan 31 22:00 /home
```

Первый символ в длинном списке - это тип файла. Вы интерпретируете так:

- - это обычный файл.
- d - это каталог.
- l - это программная ссылка.
-

Остальные символы обозначают аппаратные устройства (b и c) или другие файлы специального назначения (p и s).

Модель контроля доступа к файлам в Linux: типы разрешений

- Следующие примеры помогут проиллюстрировать, как взаимодействуют права доступа к файлам. Для этих примеров у нас есть четыре пользователя со следующим членством в группах:

Пользователь	Членство в группе
operator1	operator1, consultant1
database1	database1, consultant1
database2	database2, operator2
contractor1	contractor1, operator2

```
○ $ ls -la
```

```
total 24
```

```
drwxrwxr-x. 2 database1 consultant1 4096 Apr 4 10:23 .
drwxr-xr-x. 10 root root 4096 Apr 1 17:34 ..
-rw-rw-r--. 1 operator1 operator1 1024 Apr 4 11:02 lfile1
-rw-r--rw-. 1 operator1 consultant1 3144 Apr 4 11:02 lfile2
-rw-rw-r--. 1 database1 consultant1 10234 Apr 4 10:14 rfile1
-rw-r-----. 1 database1 consultant1 2048 Apr 4 10:18 rfile2
```

Модель контроля доступа к файлам в Linux: типы разрешений

- 🔗 В следующей таблице рассматриваются некоторые эффекты этого набора разрешений для указанных пользователей:

Эффект	Почему это истина?
Пользователь operator1 может изменять содержимое файла rfile1 .	Пользователь operator1 является членом группы consultant1 , и эта группа имеет права на чтение и запись для файла rfile1 .
Пользователь database1 может просматривать и изменять содержимое файла rfile2 .	Пользователь database1 владеет файлом и имеет права на чтение и запись в файл rfile2 .
Пользователь operator1 может просматривать, но не изменять содержимое файла rfile2 (без его удаления и повторного создания).	Пользователь operator1 является членом группы consultant1 , и эта группа имеет доступ только для чтения к файлу rfile2 .
Пользователи database2 и contractor1 не имеют доступа к содержимому файла rfile2 .	Другие (other) разрешения применяются к пользователям database2 и contractor1 , и эти разрешения не включают разрешение на чтение или запись.

Модель контроля доступа к файлам в Linux: типы разрешений

Эффект	Почему это истина?
operator1 - единственный пользователь, который может изменять содержимое файла lfile1 (без его удаления и повторного создания).	Пользователь и группа operator1 имеют разрешение на запись в файл, другие пользователи - нет. Единственный член группы operator1 - это сам пользователь operator1 .
Пользователь database2 может изменять содержимое файла lfile2 .	Пользователь database2 не является владельцем файла и не входит в группу consultant1 , поэтому применяются разрешения назначенные третьей группе other . Они предоставляют разрешение на запись.
Пользователь database1 может просматривать содержимое файла lfile2 , но не может изменять его содержимое(без его удаления и повторного создания).	Пользователь database1 является членом группы consultant1 , и эта группа имеет разрешения только на чтение для файла lfile2 . Даже если у другого (other) есть разрешение на запись, разрешения группы имеют приоритет.

Модель контроля доступа к файлам в Linux: типы разрешений

Эффект	Почему это истина?
Пользователь database1 может удалять файлы lfile1 и lfile2 .	Пользователь database1 имеет права на запись в каталог, содержащий оба файла (обозначен символом (.)), и поэтому может удалить любой файл в этом каталоге. Это верно, даже если database1 не имеет разрешения на запись в сам файл.

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- Управление локальными учетными записями пользователей
- Управление локальными группами
- Управление паролями пользователей
- Модель контроля доступа к файлам в Linux: типы разрешений
- **Управление разрешениями на файлы**
- Управление разрешениями по умолчанию и доступом к файлам

Управление разрешениями на файлы

🔗 Управление разрешениями файловой системы из командной строки

Изменение разрешений символьным методом командой **chmod**:

○ `chmodWhoWhatWhichfile|directory`

- Кто **u**, **g**, **o**, **a** (для пользователя, группы, другого, всех)
- Что делает **+**, **-**, **=** (для добавления, удаления, точной установки)
- Что определяет **r**, **w**, **x** (для чтения, записи, выполнения)

В символическом методе изменения прав доступа к файлам используются буквы для обозначения различных групп разрешений: **u**(**user**) для пользователя, **g** для группы (**group**), **o** для прочих (**other**) и **a** для всех (**all**).

Управление разрешениями на файлы

Примеры:

- Удалить права на чтение и запись для группы и других файлов в file1:

```
$ chmod go-rw file1
```

- Добавить разрешение на выполнение для всех в file2:

```
$ chmod a+x file2
```

```
-p s    exclude(^)|select PIDs          -S [t] t second stat timeout (15)
-T qs TCP/TPI Q,St (s) info
-g [s]  exclude(^)|select and print process group IDs
-i i    select by IPv[46] address: [46][proto][@host|addr][:svc_list|port_list]
+|-r [t[m<fmt>]] repeat every t seconds (15); + until no files, - forever.
        An optional suffix to t is m<fmt>; m must separate t from <fmt> and
        <fmt> is an strftime(3) format for the marker line.
-s p:s  exclude(^)|select protocol (p = TCP|UDP) states by name(s).
-u s    exclude(^)|select login|UID set s
-x [fl] cross over +d|+D File systems or symbolic Links
names select named files or files on named file systems
Anyone can list all files; /dev warnings disabled; kernel ID check disabled.
metal@metal /tmp/csv $ lsof -i
COMMAND  PID  USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
clock-app 2659 metal 23u IPv4 155897 0t0 TCP metal:52210->208-44-23-24.dia.static.qwest.net:www (ESTABLISHED)
clock-app 2659 metal 24u IPv4 155899 0t0 TCP metal:52211->208-44-23-24.dia.static.qwest.net:www (ESTABLISHED)
ssh      4500 metal 3u  IPv4 27187 0t0 TCP metal:59889->50-56-126-172.static.cloud-ips.com:ssh (ESTABLISHED)
firefox-b 21807 metal 31u IPv4 156111 0t0 TCP metal:34022->yx-in-f19.1e100.net:https (ESTABLISHED)
firefox-b 21807 metal 56u IPv4 156048 0t0 TCP metal:39026->yx-in-f83.1e100.net:https (ESTABLISHED)
firefox-b 21807 metal 57u IPv4 160841 0t0 TCP metal:54554->scl2-sync.services.mozilla.com:https (CLOSE_WAIT)
pogoplugf 22052 metal 4u  IPv4 99728 0t0 TCP metal:55856->38.126.11.28:https (CLOSE_WAIT)
pogoplugf 22052 metal 5u  IPv4 99732 0t0 UDP localhost.localdomain:59815
pogoplugf 22052 metal 6u  IPv4 160391 0t0 TCP metal:52291->38.126.11.28:www (CLOSE_WAIT)
```

Управление разрешениями на файлы

Изменение разрешений с помощью числового метода.

В приведенном ниже примере символ **#** представляет собой цифру:

```
○ chmod ### file|directory
```

- Каждая цифра представляет разрешения для уровня доступа: пользователь, группа, другой.
- Цифра рассчитывается путем сложения чисел для каждого разрешения, которое вы хотите добавить: 4 для чтения, 2 для записи и 1 для выполнения.

В трехзначном восьмеричном (числовом) представлении разрешений каждая цифра обозначает один уровень доступа слева направо: пользователь, группа и другие. Чтобы определить каждую цифру:

1. Начните с 0.
2. Если для этого уровня доступа должно быть разрешение на чтение, добавьте 4.
3. Если должно быть разрешение на запись, добавьте 2.
4. Если должно быть разрешение на выполнение, добавьте 1.

Управление разрешениями на файлы

Изучите разрешения `-rwxr-x---`. Для пользователя `rwx` рассчитывается как $4 + 2 + 1 = 7$. Для группы `r-x` рассчитывается как $4 + 0 + 1 = 5$, а для других пользователей `---` представляется с `0`. Сложив эти три вместе, числовое представление этих разрешений будет `750`. Этот расчет может быть выполнен и в обратном направлении. Посмотрите на разрешения `640`. Для прав пользователя `6` представляет чтение (`4`) и запись (`2`), что отображается как `rw-`. Для групповой части `4` включает только чтение (`4`) и отображается как `r--`. Значение `0` для других не дает разрешений (`---`), а последний набор символических разрешений для этого файла - `-rw-r-----`. Опытные администраторы часто используют числовые разрешения, потому что они короче для ввода и произношения, но при этом дают полный контроль над всеми разрешениями.

- Установите разрешения на чтение и запись для пользователя, разрешение на чтение для группы и другие для файла `samplefile`:

```
○ $ chmod 644 samplefile
```

- Установите разрешения на чтение, запись и выполнение для пользователя, права на чтение и выполнение для группы и отсутствие разрешений для других в `samplendir`:

```
○ $ chmod 750 samplendir
```

Управление разрешениями на файлы

🔗 Изменение файла и каталога, владельца пользователя или группы.

Владение файлом можно изменить с помощью команды **chown** (сменить владельца (**change owner**)). Например, чтобы предоставить право собственности на файл **test_file** пользователю- **student**, используйте следующую команду:

- # chown student test_file

chown можно использовать с параметром **-R** для рекурсивного изменения владения всем деревом каталогов. Следующая команда предоставляет студенту право владения **test_dir** и всеми файлами и подкаталогами в нем:

- # chown -R student test_dir

Команду **chown** также можно использовать для изменения группового владения файлом, поставив перед именем группы двоеточие (:).

Например, следующая команда изменяет группу **test_dir** на **admin**:

- # chown :admin test_dir

Управление разрешениями на файлы

Изменение файла и каталога, владельца пользователя или группы.

Команду **chown** также можно использовать для одновременной смены владельца и группы с помощью синтаксиса **owner:group**.

Например, чтобы изменить владельца файла **test_dir** на **visitor** и группу на **guests**, используйте следующую команду:

```
○ # chown visitor:guests test_dir
```

Вместо использования **chown** некоторые пользователи меняют владельца группы с помощью команды **chgrp**. Эта команда работает также, как **chown**, за исключением того, что она используется только для изменения владельца группы, а двоеточие (:) перед именем группы не требуется.

Управление разрешениями по умолчанию и доступом к файлам

 Специальные разрешения составляют четвертый тип разрешений в дополнение к базовым типам пользователей, групп и другим типам.		
Специальное разрешение	Влияние на файлы	Влияние на каталоги
u+s (suid)	Файл выполняется от имени пользователя, владеющего файлом, а не от имени пользователя, запустившего файл.	Нет эффекта.
g+s (sgid)	Файл выполняется как группа, владеющая файлом.	Для файлов, вновь созданных в каталоге, устанавливается владелец группы, совпадающий с владельцем группы в каталоге.
o+t (sticky)	Нет эффекта.	Пользователи с правом записи в каталог могут удалять только те файлы, которые им принадлежат; они не могут удалить файлы, принадлежащие другим пользователям.

Управление разрешениями на файлы

🔗 Специальные разрешения.

Разрешение **setuid** для исполняемого файла означает, что команды выполняются от имени пользователя, владеющего файлом, а не от пользователя, выполнившего команду. Один из примеров команда **passwd**:

```
○ $ ls -l /usr/bin/passwd
```

```
-rwsr-xr-x. 1 root root 35504 Jul 16 2010 /usr/bin/passwd
```

Специальное разрешение **setgid** для каталога означает, что файлы, созданные в каталоге, наследуют свое групповое владение от каталога, а не от создавшего его пользователя.

```
○ $ ls -ld /run/log/journal
```

```
drwxr-sr-x. 3 root systemd-journal 60 May 18 09:15 /run/log/journal
```

Если **setgid** установлен для исполняемого файла, команды выполняются от имени группы, которой принадлежит этот файл, а не от пользователя, запустившего команду, аналогично **setuid**. Одним из примеров является команда **locate**:

```
○ $ ls -ld /usr/bin/locate
```

```
-rwx--s--x. 1 root slocate 47128 Aug 12 17:17 /usr/bin/locate
```

Управление разрешениями на файлы

🔗 Специальные разрешения.

липкий бит (**sticky bit**) для каталога устанавливает специальное ограничение на удаление файлов. Только владелец файла (и **root**) может удалять файлы в каталоге. Пример каталог **/tmp**:

```
○ $ ls -ld /tmp
```

```
drwxrwxrwt. 39 rootroot 4096 Feb 8 20:52 /tmp
```

🔗 Установка специальных разрешений.

- Символически: `setuid = u+s`; `setgid = g+s`; `sticky = o+t`
- Численно: `setuid = 4`; `setgid = 2`; `sticky = 1`

Добавьте бит `setgid` в каталог:

```
○ # chmod g+s directory
```

Установите бит **setgid** и добавьте разрешения на чтение/запись/выполнение для пользователя и группы без доступа для прочих в каталоге:

```
○ # chmod 2770 directory
```

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- Управление локальными учетными записями пользователей
- Управление локальными группами
- Управление паролями пользователей
- Модель контроля доступа к файлам в Linux: типы разрешений
- Управление разрешениями на файлы
- **Управление разрешениями по умолчанию и доступом к файлам**

Управление разрешениями на файлы

Разрешения на файл по умолчанию.

Разрешения по умолчанию зависят от:

- Создается обычный файл или каталог.
- **Umask** (от англ. user file creation mode mask — маска режима создания пользовательских файлов) — функция среды POSIX, изменяющая права доступа, которые присваиваются новым файлам и каталогам по умолчанию.

Команда **umask** без аргументов отобразит текущее значение **umask** оболочки:

○ `$ umask`

```
0002
```

Системные значения **umask** по умолчанию для пользователей оболочки Bash определены в файлах `/etc/profile` и `/etc/bashrc`. Пользователи могут переопределить системные значения по умолчанию в файлах `.bash_profile` и `.bashrc` в своих домашних каталогах.

Управление разрешениями на файлы

Разрешения на файл по умолчанию.

Пример `umask`:

umask влияет на разрешения файлов и каталогов. Посмотрите на разрешения **umask** по умолчанию для файлов и каталогов в текущей оболочке. Владелец и группа имеют права на чтение и запись файлов, а остальные настроены на чтение. Владелец и группа имеют разрешения на чтение, запись и выполнение в каталогах. Для прочих единственное разрешение для чтения:

○ `$ umask`

`0002`

○ `$ touchdefault.txt`

○ `$ ls -l default.txt`

`-rw-rw-r--. 1 user user 0 May 9 01:54 default.txt`

○ `$ mkdir default`

○ `$ ls -ld default`

`drwxrwxr-x. 2 user user 0 May 9 01:54 default`

Управление разрешениями на файлы

Разрешения на файл по умолчанию.

Пример `umask`:

Если установить значение `umask` равным 0, права доступа к файлам для других файлов изменятся с чтения на чтение и запись. Разрешения каталога для других изменений от чтения и выполнения до чтения, записи и выполнения:

- `$ umask 0`

- `$ touch zero.txt`

- `$ ls -l zero.txt`

```
-rw-rw-rw-. 1 user user 0 May 9 01:54 zero.txt
```

- `$ mkdir zero`

- `$ ls -ld zero`

```
drwxrwxrwx. 2 user user 0 May 9 01:54 zero
```

Управление разрешениями на файлы

Разрешения на файл по умолчанию.

Пример `umask`:

Чтобы применить маску на все права доступа к файлам и каталогам для прочих, установите значение `umask`, равным `007`:

○ `$ umask 007`

○ `$ touch seven.txt`

○ `$ ls -l seven.txt`

```
-rw-rw----. 1 user user 0 May 9 01:55 seven.txt
```

```
$ mkdir seven
```

○ `$ ls -ld seven`

```
drwxrwx---. 2 user user 0 May 9 01:54 seven
```

Управление разрешениями на файлы

Разрешения на файл по умолчанию.

Пример `umask`:

`Umask 027` гарантирует, что новые файлы будут иметь права на чтение и запись для пользователя и права на чтение для группы. Новые каталоги имеют права чтения и записи для группы и не имеют прав для прочих:

- `$ umask 027`

- `$ touch two-seven.txt`

- `$ ls -l two-seven.txt`

```
-rw-r-----. 1 user user 0 May 9 01:55 two-seven.txt
```

- `$ mkdir two-seven`

- `$ ls -ld two-seven`

```
drwxr-x---. 2 user user 0 May 9 01:54 two-seven
```


Управление разрешениями на файлы

Разрешения на файл по умолчанию.

Пример `umask`:

Как **root**, вы можете изменить значения по умолчанию, добавив сценарий запуска оболочки с именем `/etc/profile.d/localumask.sh`:

```
○ # cat /etc/profile.d/local-umask.sh
```

```
○ # Overrides default umask configuration
```

```
if [ $UID -gt 199 ] && [ "`id -gn`" = "`id -un`" ]; then
```

```
○ umask 007
```

```
else
```

```
○ umask 022
```

```
○ Fi
```

Если вы просто хотите установить **umask** для всех равным **022**, вы можете создать этот файл со следующим содержимым:

```
○ # Overrides default umask configuration
```

```
umask 022
```

В данной л были рассмотрены следующие темы:

- Концепция пользователей и групп в Linux
- Получение привилегий суперпользователя
- Управление локальными учетными записями пользователей
- Управление локальными группами
- Управление паролями пользователей
- Модель контроля доступа к файлам в Linux: типы разрешений
- Управление разрешениями на файлы
- Управление разрешениями по умолчанию и доступом к файлам